# AhamX Bodhi

Where I Becomes Infinite

# Self-Attention Mechanism

Module 2.2: Transformer Architecture Fundamentals

Prof. Sashikumaar Ganesan | IISc Bengaluru

# Session Overview

## What You Will Learn

- Why attention replaces recurrence in sequence models
- The Query–Key–Value (Q, K, V) abstraction
- Scaled dot-product attention computation
- How context is built from similarity scores

## Concept at a Glance

- **Duration:** 10 minutes
- **Bloom's Level:** B2 – Understand
- **Difficulty:** L2 – Intermediate-Beginner
- **Module:** 2.2 Transformer Architecture Fundamentals
- **Prerequisite:** What is a Transformer?

# Learning Objectives

## By the End of This Session You Will Be Able To

1. **Explain** why self-attention is needed and how it differs from RNNs
2. **Define** the Query, Key, and Value matrices and their roles
3. **Describe** the scaled dot-product attention formula step by step
4. **Interpret** attention scores as contextual relevance weights
5. **Identify** how self-attention enables parallelism during training

# Prerequisites and Connections

## You Should Already Know

- What a Transformer model is (Module 2.1)
- What token embeddings represent
- Autoregressive generation and next-token prediction
- Basic matrix multiplication intuition

## This Concept Unlocks

- Multi-Head Attention (Module 2.3)
- Positional Encoding (Module 2.4)
- Decoder-Only Architecture (Module 2.6)
- Fine-Tuning and LoRA (Module 7)
- KV-Cache and Inference Optimization (Module 9)

# The Problem Self-Attention Solves

## Limitations of Recurrent Neural Networks (RNNs)

- **Sequential bottleneck:** Token $t$ cannot be processed until token $t-1$ is done
- **Vanishing gradients:** Long-range dependencies are lost over many steps
- **No parallelism:** Training on long sequences is extremely slow
- **Fixed memory:** A single hidden state must carry all prior context

Self-attention breaks the sequential constraint by allowing **every token to directly attend to every other token** in a single parallel operation — regardless of distance.

# The Core Intuition --- A Library Analogy

### Think of a Library Search System

- **Query (Q):** The question you bring to the library — “what am I looking for?”
- **Key (K):** The index card on each book’s spine — “what does this book contain?”
- **Value (V):** The actual content inside the book — “what information do I get?”
- **Attention score:** How well your query matches each book’s index card
- **Output:** A weighted blend of book content, proportional to relevance

In a sentence, each word simultaneously *queries* all other words, finds the most *relevant* ones (via key matching), and aggregates their *values* to form a richer, contextualised representation.

# Step 1 --- Projecting Tokens into Q, K, V

## From Token Embeddings to Q, K, V Matrices

Given a sequence of $n$ tokens with embedding dimension $d_{\text{model}}$, three learned weight matrices $\mathbf{W}^Q, \mathbf{W}^K, \mathbf{W}^V \in \mathbb{R}^{d_{\text{model}} \times d_k}$ project the input matrix $\mathbf{X} \in \mathbb{R}^{n \times d_{\text{model}}}$ into:

$$\mathbf{Q} = \mathbf{X}\mathbf{W}^Q, \quad \mathbf{K} = \mathbf{X}\mathbf{W}^K, \quad \mathbf{V} = \mathbf{X}\mathbf{W}^V$$

## Key Insight

- All three matrices are derived from the **same input** $\mathbf{X}$ — hence "*self*-attention"
- $\mathbf{W}^Q, \mathbf{W}^K, \mathbf{W}^V$ are **learned** during training, not fixed
- $d_k$ is typically $d_{\text{model}}/h$ where $h$ is the number of attention heads

# Step 2 --- Scaled Dot-Product Attention

## The Core Formula

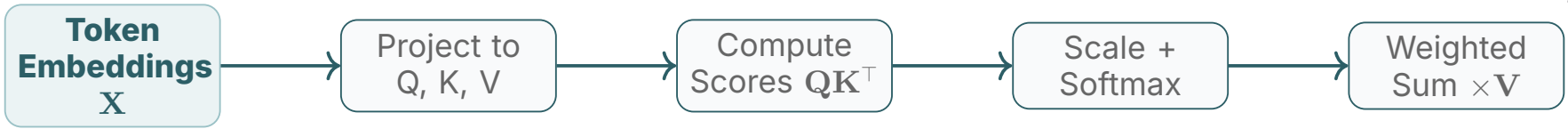
$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}}\right)\mathbf{V}$$

- $\mathbf{Q}\mathbf{K}^\top \in \mathbb{R}^{n \times n}$ — raw similarity score between every pair of tokens
- $\sqrt{d_k}$ scaling — prevents dot products from growing too large, stabilising gradients
- $\text{softmax}(\cdot)$ — converts scores to a probability distribution (rows sum to 1)
- Multiply by $\mathbf{V}$ — weighted sum of value vectors, one output per token

## Why Divide by $\sqrt{d_k}$?

For large $d_k$, dot products grow large in magnitude, pushing softmax into regions of very small gradients. Dividing by $\sqrt{d_k}$ keeps variance near 1.

# Self-Attention: Four Steps at a Glance




## Reading the Pipeline

- **Project:** Multiply $\mathbf{X}$ by learned $\mathbf{W}^Q$, $\mathbf{W}^K$, $\mathbf{W}^V$
- **Score:** Compute $n \times n$ dot-product matrix (token-to-token similarity)
- **Normalise:** Divide by $\sqrt{d_k}$, apply softmax row-wise
- **Aggregate:** Weight-and-sum value vectors to produce contextual output

# Concrete Example --- Resolving Ambiguity with Context

Sentence: “I went to the **bank** to deposit money”

- Token **“bank”** generates a Query that attends to all other tokens
- **“deposit”** and **“money”** produce high attention scores (strong key match)
- **“went”** and **“I”** receive low attention scores (weak key match)
- The softmax-weighted sum of Value vectors for “bank” now encodes **financial institution**, not river bank

This context-sensitive representation is what enables Transformers to handle polysemy, coreference, and long-range dependencies that RNNs struggle with.

# Attention Score Matrix --- What It Represents

## The $n \times n$ Attention Weight Matrix

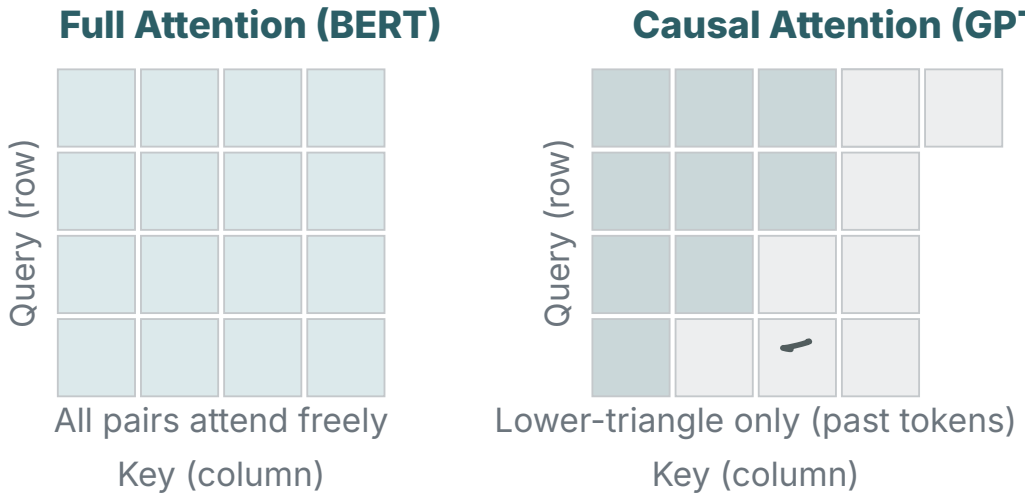
For a sequence of $n$ tokens, the attention matrix $\mathbf{A} \in \mathbb{R}^{n \times n}$ has:

- Row $i$ = distribution over all tokens that token $i$ attends to
- Entry $a_{ij}$ = how much token $i$ weighs information from token $j$
- Each row sums to 1 after softmax (a proper probability distribution)

## Causal Masking in Decoder-Only Models

- GPT-style models add a **causal mask**: token $i$ can only attend to positions $j \leq i$
- Upper-triangular entries are set to $-\infty$ before softmax (zeroed out after)
- This enforces left-to-right generation: no token can "see the future"

# Full Attention vs. Causal (Masked) Attention




Encoder models (BERT) use **full bidirectional** attention; decoder-only models (GPT, Llama, Claude) use **causal** attention during generation.

# Computational Complexity of Self-Attention

## Cost Analysis

- Computing $\mathbf{QK}^\top$ costs $O(n^2 d_k)$ — quadratic in sequence length $n$
- For a 4096-token context: $\sim 4096^2 \approx 16.7$M pairwise comparisons per head
- Memory for the attention matrix: $O(n^2)$ — the main bottleneck at long context
- Mitigations: Flash Attention, Sliding Window Attention, Linear Attention approximations

## Quadratic Scaling is Why Context Windows Are Expensive

- Doubling context length **quadruples** the attention computation and memory
- Extending from 4K to 128K tokens requires significant engineering effort
- Flash Attention (Dao et al. 2022) reduces memory to $O(n)$ via tiled recomputation

H.B Parmets $\simeq (6 \times 4B)$ FLOP.
Compute cost

# Self-Attention vs. Recurrent Networks --- Comparison

| Property | Self-Attention | RNN / LSTM |
|---|---|---|
| Parallelism | Full (all tokens at once) | Sequential (token by token) |
| Long-range dependencies | Direct (one operation) | Indirect (many steps) |
| Gradient path length | $O(1)$ | $O(n)$ |
| Memory per step | $O(n^2)$ attention matrix | $O(1)$ hidden state |
| Training speed | Fast (GPU-friendly) | Slow (inherently sequential) |
| Context window | Bounded by $n^2$ cost | Theoretically unlimited |

# Why Self-Attention Enables Parallelism

## Training on GPUs: The Key Advantage

- RNNs require hidden state from step $t-1$ before processing step $t$
- Self-attention processes **all $n$ tokens simultaneously** as a matrix operation
- $\mathbf{QK}^\top$ and the final softmax$(\cdot)\mathbf{V}$ are dense matrix multiplications
- Dense matmuls are exactly what GPUs are optimised for (CUDA cuBLAS, Tensor Cores)
- This unlocks training on web-scale corpora in tractable time

GPT-3 (175B parameters) was trained on 300B tokens in approximately 34 days using 1024 A100 GPUs — only feasible because attention is fully parallelisable.

# Where Self-Attention Powers GenAI Applications

## Production Systems Built on Self-Attention

- **ChatGPT / GPT-4o (OpenAI):** Every reply token is generated by a stack of self-attention layers that contextualise the full conversation history
- **Claude (Anthropic):** Processes up to 200K tokens in a single self-attention context window; cross-document reasoning depends on long-range attention
- **Gemini 1.5 Pro (Google):** Extends self-attention to 1M-token windows using ring attention and mixture-of-experts variants
- **GitHub Copilot:** Attends to the full file context to suggest completions
- **Stable Diffusion / DALL-E 3:** Cross-attention between text and image tokens drives visual grounding of language descriptions

# Self-Attention in Specialised Domains

## Beyond Text: Self-Attention Across Modalities

- **Code (CodeLlama, StarCoder):** Self-attention captures variable scoping, function definitions, and cross-file dependencies in codebases
- **Proteins (AlphaFold 2):** Self-attention on amino acid sequences learns residue-to-residue contacts that determine 3D protein structure
- **Vision Transformers (ViT):** Image patches attend to each other — self-attention replaces convolutional filters for image classification
- **Speech (Whisper, SeamlessM4T):** Attention over spectrogram frames enables robust ASR across 100+ languages
- **Retrieval-Augmented Generation (RAG):** Retrieved documents are concatenated into context; self-attention integrates them with the query

# Self-Attention at Inference: The KV Cache

## Why KV Caching Matters for Production LLMs

- During autoregressive generation, past token Keys and Values are re-used each step
- Recomputing $\mathbf{K}$ and $\mathbf{V}$ for all previous tokens every step is wasteful
- **KV cache:** Store computed $\mathbf{K}$, $\mathbf{V}$ tensors in GPU memory and append only the new token
- Reduces per-step cost from $O(n^2)$ to $O(n)$ for each new token

## Real-World Impact

- KV cache is the primary GPU memory consumer in deployed LLMs
- A 70B model at 4K context needs $\sim$18 GB for KV cache alone
- Techniques like PagedAttention (vLLM) manage KV cache as virtual memory

# How Self-Attention Explains Prompt Sensitivity

## Why Prompt Wording Matters So Much

- Every token in your prompt is projected to Q, K, V vectors
- The phrasing of a word changes its embedding, which changes its K vector
- Different K vectors cause different tokens to “fire” in the attention matrix
- **Adding examples (few-shot):** New tokens enter the context and shift attention patterns
- **System prompt:** Attended to by every subsequent user and assistant token

Understanding self-attention is the mechanistic foundation for understanding **why prompt engineering works** — you are shaping the input that drives the attention mechanism.

# Common Misconceptions --- Part 1

## Do Not Confuse These

- **“Self-attention is the same as attention in seq2seq models.”**
  *No.* Classic seq2seq attention (Bahdanau 2015) computes query from decoder and keys from encoder. Self-attention has Q, K, V all from the *same* source.
- **“Softmax outputs tell you which tokens the model understands best.”**
  *No.* They represent learned relevance weights for the task — not interpretable as human-readable importance without further analysis.
- **“Higher attention score means the model thinks those words are synonyms.”**
  *No.* Attention scores encode task-relevant relationships, not semantic similarity.

# Common Misconceptions --- Part 2

## More Things to Watch Out For

- **"Self-attention gives the model memory between conversations."**
  *No.* Attention operates only within the context window of a single forward pass; there is no memory persisted across separate API calls.
- **"The $\sqrt{d_k}$ scaling is unimportant."**
  *No.* Without it, large $d_k$ causes extremely peaked softmax distributions and vanishing gradients, significantly harming training stability.
- **"Self-attention replaces all computation in a Transformer."**
  *No.* Each Transformer block also includes a Feed-Forward Network (FFN), Layer Normalisation, and Residual Connections.

# Connection to Multi-Head Attention

## Single Head vs. Multiple Heads

- A single self-attention head computes one set of $\mathbf{Q}$, $\mathbf{K}$, $\mathbf{V}$ projections
- **Multi-Head Attention (MHA):** Run $h$ heads in parallel, each with its own weight matrices, then concatenate and project the outputs
- Each head can specialise: one head may attend to syntactic structure, another to semantic relations, another to coreference

## What Multi-Head Unlocks

- Richer, multi-perspective representations per token
- Total cost roughly the same as one fat head (dimension split across heads)
- GPT-3 uses 96 heads; Llama 3 8B uses 32 heads

# Summary

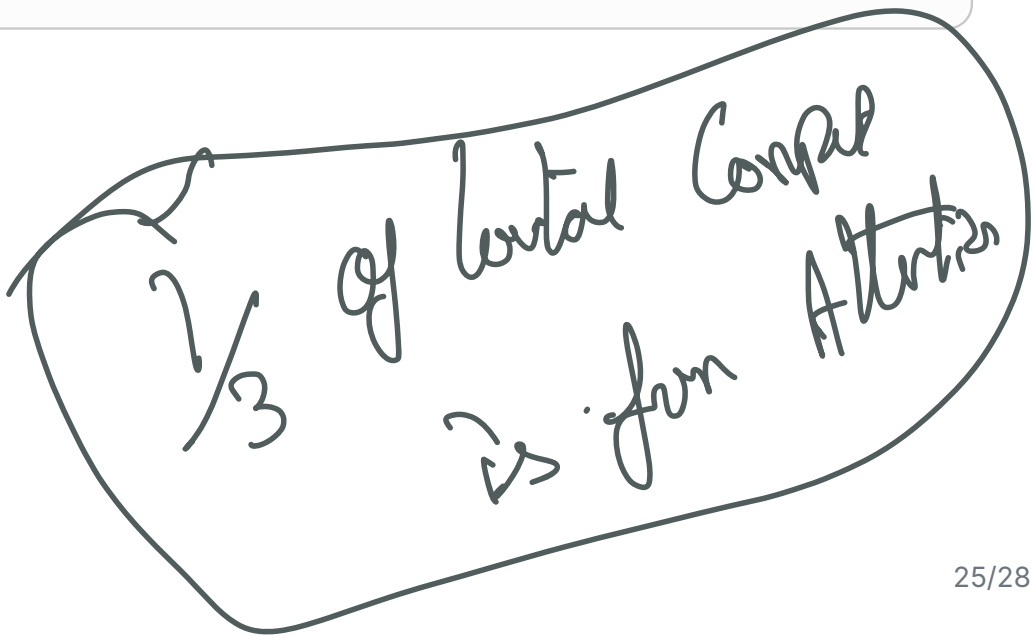
## Self-Attention Mechanism — Key Takeaways

1. Self-attention replaces recurrence by allowing **every token to attend to every other token** in parallel, solving the sequential bottleneck of RNNs
2. The **Query–Key–Value** framework: Q asks, K is matched against, V is aggregated
3. The formula $\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}(\mathbf{Q}\mathbf{K}^\top / \sqrt{d_k})\,\mathbf{V}$ is the core computation
4. **Causal masking** in decoder-only models enforces left-to-right generation
5. Quadratic $O(n^2)$ complexity motivates KV-caching and Flash Attention at inference
6. Self-attention is the architectural foundation of every major LLM deployed today

# What You Needed to Know Before This Session

## Prerequisites in the Curriculum

- **What is a Transformer?** — High-level overview of the encoder-decoder stack
- **Autoregressive Generation** — How LLMs produce tokens one at a time
- **Probability and Token Prediction** — Softmax over vocabulary distributions
- **Foundation Models Overview** — Why scale enables emergent capabilities



# What This Concept Unlocks

## Immediate Next Concepts

- Multi-Head Attention (Module 2.3)
- Positional Encoding (Module 2.4)
- Feed-Forward Networks in Transformers (Module 2.5)
- Decoder-Only Architecture (Module 2.6)

## Downstream Concepts

- LoRA: Low-Rank Adaptation (Module 7)
- KV Cache and Inference Optimisation (Module 9)
- Flash Attention (Module 9)
- Cross-Attention in RAG and Diffusion (Module 5)

# Next Steps

## Recommended Actions

- Trace through the attention formula by hand for a 3-token sequence
- Visualise attention maps using BertViz or TransformerLens
- Read Vaswani et al. (2017) “Attention Is All You Need”
- Explore the Illustrated Transformer (Jay Alammar blog)

## Coming Up in Module 2

- Multi-Head Attention — running $h$ attention heads in parallel
- Positional Encoding — how position is injected without recurrence
- Layer Normalisation and Residual Connections
- Complete Transformer Block walkthrough

# Thank You

Self-Attention Mechanism | Module 2.2 | AI-TR-FN-TH-000002